LAB 7: FUZZING FOR INPUT VALIDATION BUGS

Credit Card Rewards Parser

Student: N Sharath
SRN: PES1UG25AM806
University: PES University

Objective

Use Hypothesis-based fuzz testing to exercise three input-processing functions, identify failures caused by unexpected inputs, harden the functions, and rerun the tests.

Functions tested

1. **mask_card_number(number)** – masks a card number.
2. **parse_transaction(line)** – parses a transaction category and amount.
3. **parse_statement_total(text)** – extracts a statement total.

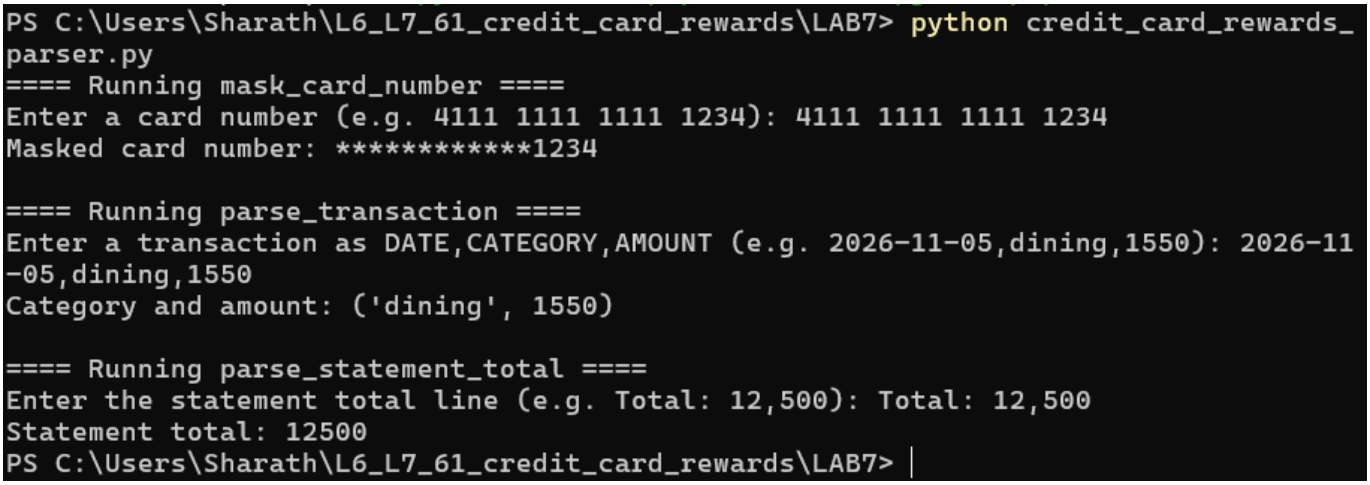
Test summary

The completed Hypothesis test file contains three no-crash properties. The recorded final run shows **3 tests passed**.

Evidence screenshots

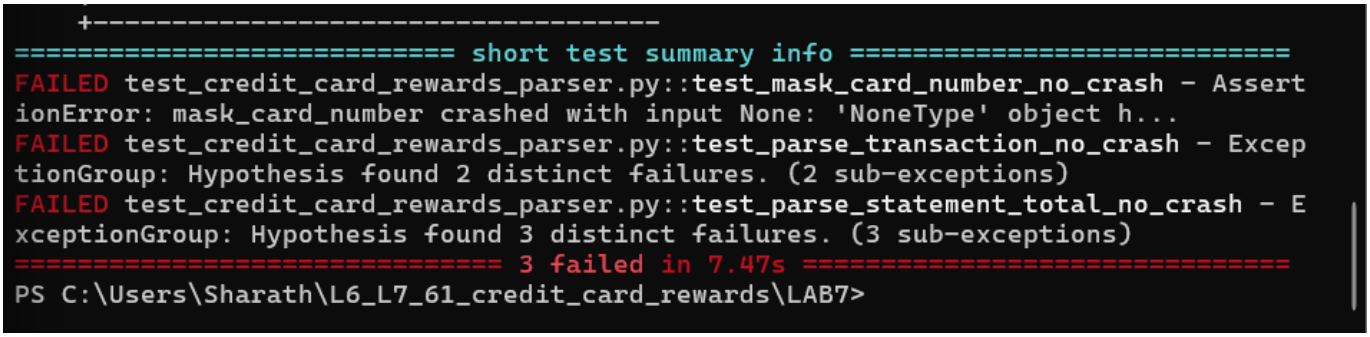
The following pages contain the four terminal screenshots captured during the lab.

SS1 – Original application run



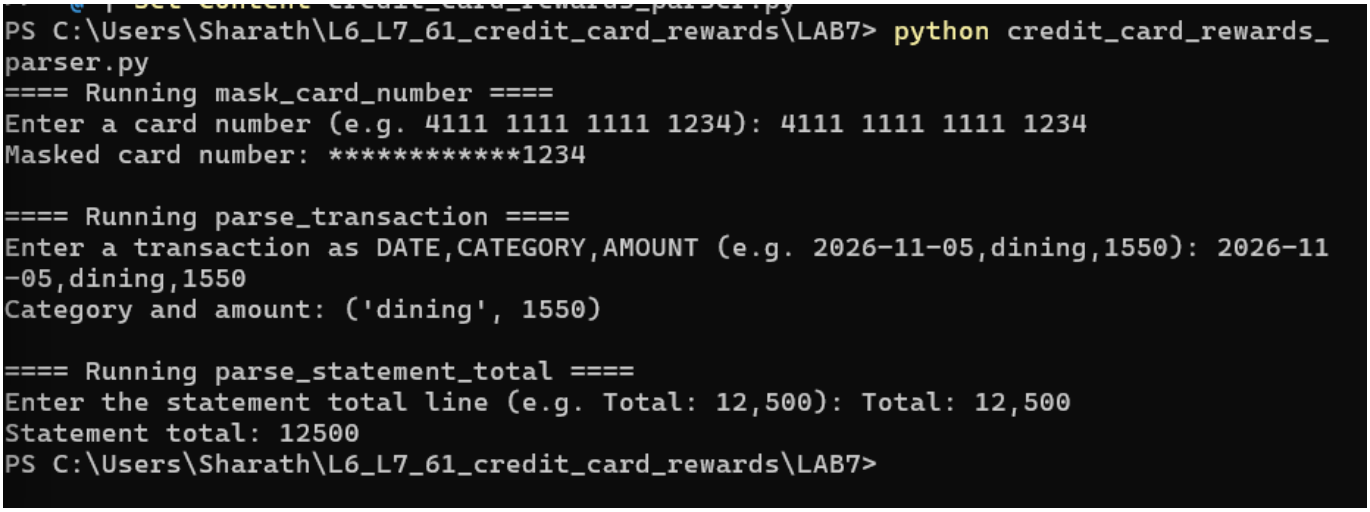
Original application execution with sample card number, transaction, and statement total inputs.

SS2 – Fuzzing failures



Hypothesis tests on the buggy implementation: all three no-crash tests failed.

SS3 – Fixed application run



Application execution after input validation and defensive handling were added.

SS4 – Fuzzing tests passed

```
PS C:\Users\Sharath\L6_L7_61_credit_card_rewards\LAB7> python -m pytest -v test_cr
edit_card_rewards_parser.py
============================= test session starts =============================
platform win32 -- Python 3.14.0, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\Sharath\Ap
pData\Local\Programs\Python\Python314\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\Sharath\L6_L7_61_credit_card_rewards\LAB7
plugins: anyio-4.16.0, hypothesis-6.170.15, cov-7.1.0
collected 3 items

test_credit_card_rewards_parser.py::test_mask_card_number_no_crash PASSED  [ 33%]
test_credit_card_rewards_parser.py::test_parse_transaction_no_crash PASSED [ 66%]
test_credit_card_rewards_parser.py::test_parse_statement_total_no_crash PASSED [10
0%]

============================== 3 passed in 1.99s ==============================
PS C:\Users\Sharath\L6_L7_61_credit_card_rewards\LAB7>
```

Final Hypothesis run: all three no-crash tests passed.

Reflection

Note: The supplied four-page handout does not contain the three reflection questions mentioned in the repository README. The following are suggested reflection prompts and answers based on the work completed; replace them if your instructor provides exact questions.

1. What is fuzz testing, and why is it useful?

Fuzz testing is a dynamic testing technique that supplies unexpected, random, or malformed inputs to a program. It is useful because it can reveal crashes and edge cases that ordinary hand-written tests may not cover.

2. What is Hypothesis, and how was it used in this lab?

Hypothesis is a property-based testing library for Python. In this lab, the **@given** decorator and text/None strategies were used to generate a range of inputs for each parser function and check that the functions did not crash.

3. What problems were found and how were they addressed?

The tests exposed exceptions when the functions received inputs such as **None**, malformed transaction records, or statement text without a valid amount. The functions were hardened with input-type checks, format validation, and safe handling of invalid numeric values. The final fuzz test run reported three passing tests.

Conclusion

The three functions were tested with Hypothesis-generated inputs and updated to handle unexpected input more safely. The final recorded test run completed with 3 passed.